

# 9 System Architecture

## 9.1 System Environment

We mentioned several times in this book that OPC UA is designed in a generic manner and can therefore be applied in a diverse range of applications running at various locations within an organization's network. Figure 9.1 shows an example of an environment in which OPC UA is used in various ways. In this scenario, OPC UA servers are running on controllers of the Control Network, on Batch systems in the Operations Network, and are applied for MES application as a part of the production planning. Furthermore, an ERP system uses an OPC UA client as an interface for consuming services in the Corporate Network. In addition, there are not only different applications involved but also different platforms. The controllers require real-time operating systems, the Batch systems as well as the MES might be Windows-based and the ERP system might be deployed on UNIX platforms. Besides the possibility to run OPC UA on different platforms, it can also be used for applying different architectural concepts at system level. In our example, several architectural concepts can be identified such as redundancy (Batch servers), server-chaining (MES and Batch), or server aggregation (Batch and OPC UA servers running on the controllers).

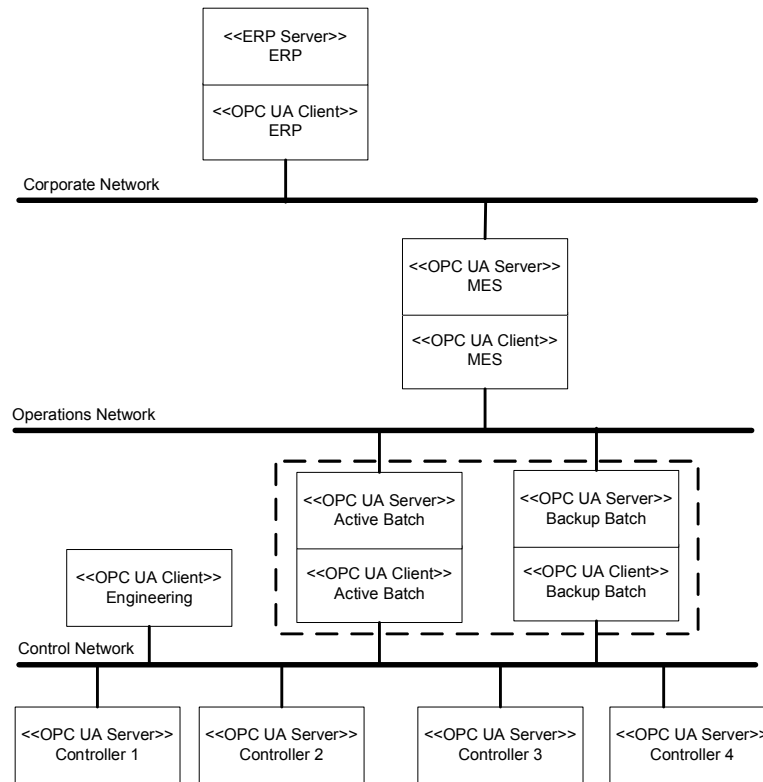
The following sections describe how OPC UA can be used to apply these and other concepts at system level.

## 9.2 Basic Architecture Patterns

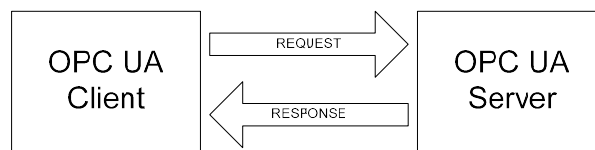
This section introduces different architectural patterns for OPC UA systems. Such patterns are either used for structuring systems and applications or used for solving certain design problems. In the following, four basic patterns, Client–Server, Chained Server, Server-to-Server Communication, and Aggregating Servers, are described that can be applied in different OPC UA-specific scenarios.

### 9.2.1 Client-Server

The first pattern—the Client–Server pattern – is the most common one and represents the basic OPC UA communication pattern (Fig. 9.2). There are two roles defined in this scenario: a server offering a service and a client consuming that service to fulfill certain tasks. The communication between them is defined by a contract they both agree upon. This means for the present context that an OPC UA client sends a well-formed request message to an OPC UA server, which answers to the request with an appropriate response message.



**Fig. 9.1** Example for an OPC UA system environment

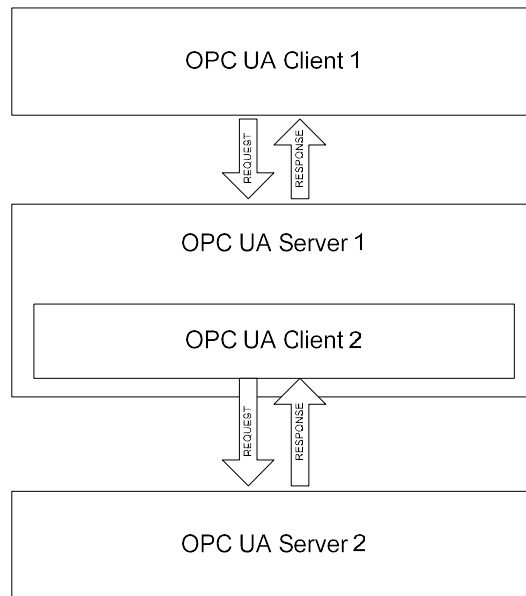


**Fig. 9.2** Client-Server pattern

### 9.2.2 Chained Server

In the Chained-Server pattern depicted in Fig. 9.3 three entities are involved: a common OPC UA client (OPC UA Client 1), an OPC UA server (OPC UA Server 1) with an embedded client (OPC UA Client 2), and a common OPC UA server (OPC UA Server 2). The OPC UA Client 1 exchanges messages with OPC UA Server 1 and the embedded OPC UA Client 2 communicates with OPC UA Server 2.

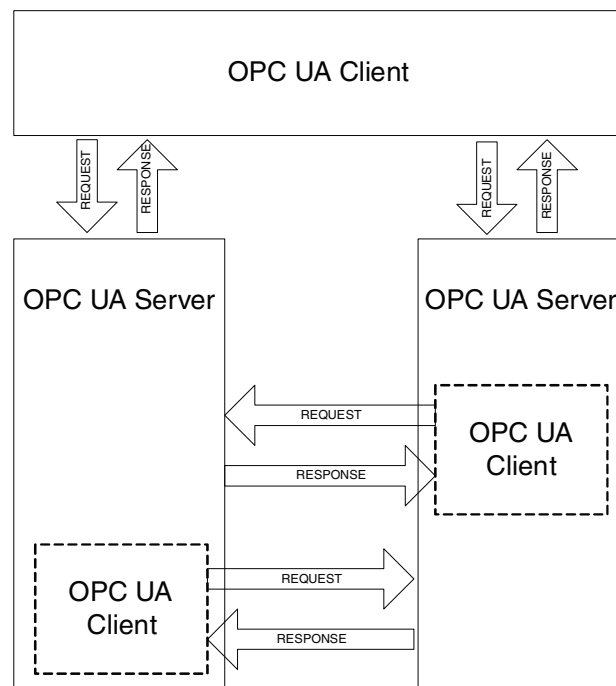
There are several use cases and scenarios in which the application of such a pattern is useful. One possible scenario could be to use the chaining server as a gateway. If, for example, OPC UA Client 1 only supports HTTP(s) as transport protocol and the server it intends to talk with resides in a network segment in which a firewall restricts the access to TCP combined with a special port, then a chaining server could act as a gateway in-between. Thereby, it translates HTTP(s) requests coming from the client to TCP and responses in TCP to HTTP(s) since it acts as client to OPC UA Server 2 and as a server to OPC UA Client 1.



**Fig. 9.3** Chained Server pattern

### 9.2.3 Server-to-Server Communication

The term Server-to-Server communication sounds somewhat contradictory, since we know from the basic Client–Server pattern that a server provides services that are only consumed by clients. But how can servers communicate with each other? This can be achieved by embedding a client into a server. This concept is quite



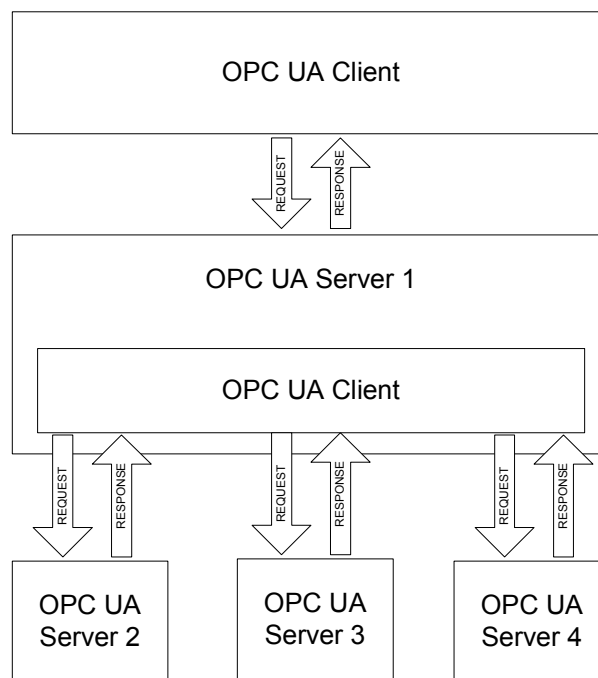
**Fig. 9.4** Server-to-Server pattern

similar to the Chained-Servers pattern introduced in the Sect. 9.2.2, with the exception that both servers have embedded clients, which enables both sides to initiate the communication (i.e., both sides can send request messages). This scenario is illustrated in Fig. 9.4. Note that although this concept is named Server-to-Server communication, this does not mean that only servers can communicate with each other. Of course, non-embedded OPC UA clients can talk with the servers as well.

But why should servers communicate with each other? One very typical use case is server redundancy. To provide the same data to clients in a failover case, redundant servers have to replicate their data especially when both servers are running and in operational mode (e.g., hot-failover concept described in Sect. 9.3).

### 9.2.4 Aggregating Server

Finally, the last of the four patterns described in this chapter is the concept of Aggregating Servers. This pattern is also similar to the Chaining-Server concept. In Fig. 9.5, an example for that concept is given. The common OPC UA Client talks with OPC UA Server 1 containing an embedded OPC UA Client. This embedded client accesses data provided by multiple other OPC UA servers. The data retrieved from these servers is prepared or processed in a special way by the intermediate server before a response is sent back to the common client.



**Fig. 9.5** Aggregating Server pattern

Such a pattern can, for example, be applied in the field of MES. In such a scenario, the common OPC UA client can be used for executing and supervising production requests. A production request contains among others information about the type and the quantity of the product that has to be manufactured. OPC UA Server 1 processes the request and distributes subtasks to the underlying servers. Each of the aggregated servers is thereby responsible for a defined part of the production process. Once they have completed their tasks, they return the results to OPC UA Server 1, which composes a response for the common client. Before sending the response to the common client, the data obtained from the aggregated servers can be prepared in a special way for the client. For example, the overall status of the request or some statistical data like number of occurred errors can be returned.

The main difference between an aggregating server and a chaining server is that the chaining server just passes the data of the underlying server(s), while an aggregating server typically concentrates the information of the underlying server(s).

## 9.3 Redundancy

Redundancy in general is the existence of multiple critical components of a system in order to increase the reliability. If an error occurs in one of the components, then another one is used instead.

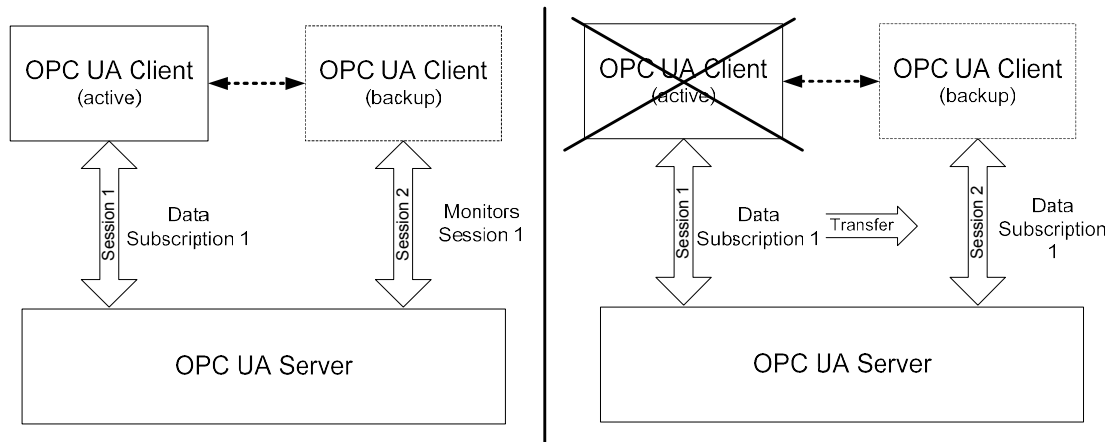
In OPC UA, redundancy is based on the existence of duplicate client or server applications and can be achieved by using special data structures and services of OPC UA. In the following, we distinguish between client and server redundancy.

### 9.3.1 *Client Redundancy*

Client redundancy is needed in environments in which, for example, a continuous supervisory of a production process is needed. Another example is when an OPC UA server aggregates data from underlying servers to perform special calculations (Sect. 9.2.4). OPC UA supports these types of redundancy by applying the TransferSubscriptions Service in combination with monitoring client information residing in the server's Address Space.

Figure 9.6 illustrates an example of how this can work. Let us assume that we have an active OPC UA client with running data subscriptions and a backup client. This backup client monitors the session information of the active client in the server address space in the same way any other data is monitored in OPC UA. Once the active client fails and the status of the session changes in the address space, the backup client uses the service TransferSubscriptions to get all running subscriptions from the active client. Subscriptions can survive sessions since their lifetime is independent from the session lifetime. The server must thereby buffer data to be sent to the client during this failover process to avoid losing data. This

mechanism requires the backup client to have knowledge about the SessionId for monitoring the session and in addition the SubscriptionId for transferring the subscriptions of the active client. However, there is no standard way specified by OPC UA for exchanging this information between those clients.



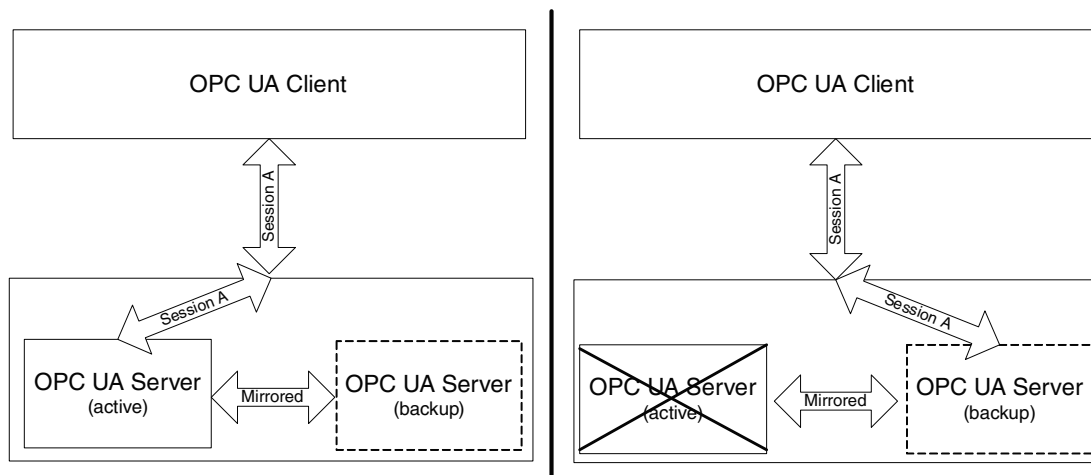
**Fig. 9.6** Example of client redundancy

### 9.3.2 Server Redundancy

Server redundancy can be further differentiated in transparent server redundancy and nontransparent server redundancy.

#### 9.3.2.1 Transparent Server Redundancy

In the first approach, server redundancy is handled transparently to the client. This means that in a failover case the client does not realize that an error occurred and



**Fig. 9.7** Transparent server redundancy

it does not need to do anything to continue performing its tasks. However, the server has the full responsibility to ensure that the client can access its required data. This means that the redundant servers have to be mirrored. They have to have exactly the same data and session information. Figure 9.7 shows an example of how this can look like in the context of OPC UA. If the active server is not available anymore, further requests are redirected to the identical backup server.

Although clients do not have to do anything special accessing a redundant server, the information that they are accessing a redundant server can be found in the Address Space. To achieve requirements from FDA,<sup>1</sup> the server also exposes an Id that uniquely identifies the server in the redundant set of servers.

### 9.3.2.2 Nontransparent Server Redundancy

In contrast to the above described transparent server redundancy concept, the nontransparent server redundancy requires some actions of the client to continue its work in the case when the server fails. For this type of redundancy, OPC UA specifies several failover modes defining for each case the role of the backup server. Depending on the selected failover mode at the client side, different actions to support redundancy have to be performed. Table 9.1 describes each mode and the roles of the backup server as well as the required actions for the client side.

### 9.3.2.3 Nontransparent Server Redundancy Approaches

To perform the failover, the client has to create a new session to a backup server and transfer the subscription data of the previous session to the new session. For doing that, OPC UA proposes two approaches: by duplicating subscriptions or by using the TransferSubscriptions Service. Both ways are described later. Since this mechanism can be reused by any type of OPC UA client, it makes sense to encapsulate this functionality into a separated component (Failover Proxy).

The first approach (Fig. 9.8) requires only actions of the Failover Proxy residing on the client side. In this case, it acts as proxy and creates a connection on both active and backup server. Furthermore, all subscriptions created on the active server are created on the backup as well while sampling or reporting is only enabled on the active one. Other service requests such as Read or Write are only forwarded. If, in a failover case, the active server is not accessible, then the proxy component enables sampling or reporting on the backup server.

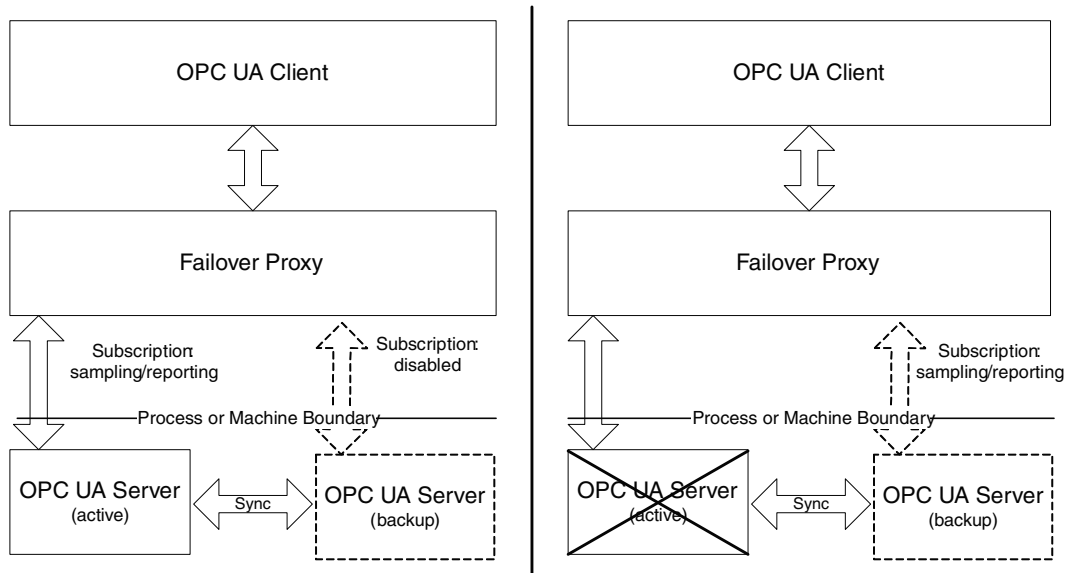
---

<sup>1</sup> The U.S. Food and Drug Administration (FDA) define requirements for companies developing applications for the consumer industry in the United States

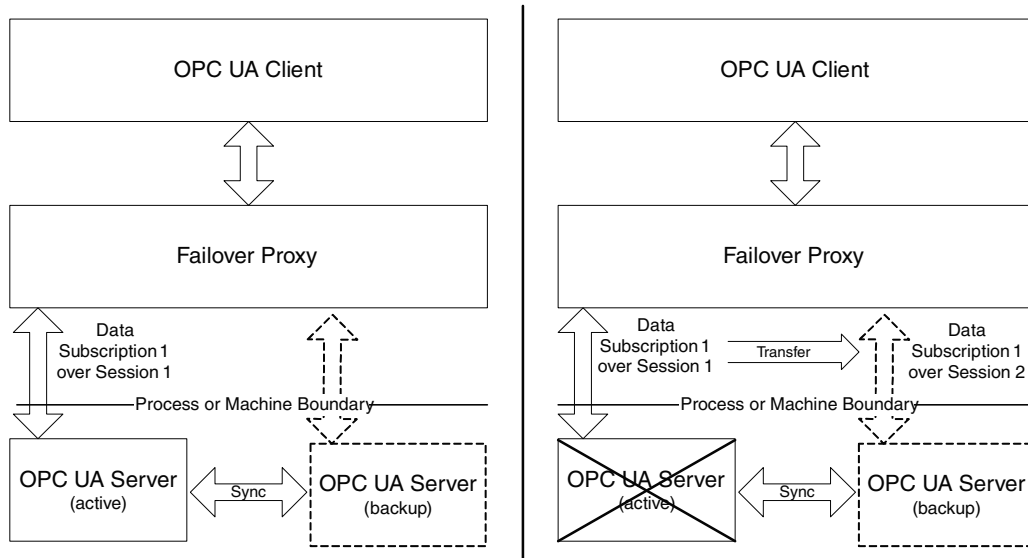
**Table 9.1** Failover modes and client side actions

| Failover mode | Role of the backup server                                                                                                                                                                                                                                                  | Client side actions                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cold          | Backup server is running but not active. Once the active server fails the backup is activated.                                                                                                                                                                             | <b>On initial connection:</b> <ol style="list-style-type: none"> <li>1. Nothing to do</li> </ol> <b>At Failover:</b> <ol style="list-style-type: none"> <li>1. Connect to backup server</li> <li>2. Create subscriptions and add monitored items</li> <li>3. Activate sampling on the subscriptions</li> <li>4. Activate reporting of notifications</li> </ol>                                                    |
| Warm          | Backup server is running and active but it cannot connect to actual data points. This is applied in scenarios in which the number of connections to the underlying devices is limited. Therefore, the backup server only connects to the device in the case of a failover. | <b>On initial connection:</b> <ol style="list-style-type: none"> <li>1. Connect to both active and backup</li> <li>2. Create subscriptions and add monitored items (enable subscription on active server and disable on backup server)</li> </ol> <b>At Failover:</b> <ol style="list-style-type: none"> <li>1. Activate sampling on the subscriptions</li> <li>2. Activate reporting of notifications</li> </ol> |
| Hot           | Backup server is like the active server full operational and can be used for accessing data. When the active server breaks, then the backup server is running with a higher load since all clients previously connected to the active server move to the backup server.    | <b>On initial connection:</b> <ol style="list-style-type: none"> <li>1. Connect to both active and backup</li> <li>2. Create subscriptions and add monitored items</li> <li>3. Activate sampling on the subscriptions</li> </ol> <b>At Failover:</b> <ol style="list-style-type: none"> <li>1. Activate reporting of notifications</li> </ol>                                                                     |

The second approach requires actions of the Failover Proxy component on the client side and additional actions on the server side. The proxy on the client side only creates subscriptions on the active server. The active server mirrors all created subscriptions to the backup server. Once the active server fails, the proxy creates a new session on the backup server and uses the TransferSubscriptions Service for getting the subscriptions of the previous session. This is exemplified in Fig. 9.9.



**Fig. 9.8** Nontransparent server redundancy – approach 1



**Fig. 9.9** Nontransparent server redundancy – approach 2

## 9.4 Discovery

### 9.4.1 Why Discovery?

In large OPC UA environments, there might be scenarios in which many OPC UA servers with different endpoints are provided. Each endpoint can have different configurations regarding communication, encoding, or security as well. In addition, the servers can run at different locations (e.g., network segments or sites). The

problem for a client here is to find the server it intends to communicate with and which has an endpoint that the client is able to connect to. This is why OPC UA clients have to discover servers and inform themselves about the existing configuration options of the endpoints before they can connect to a server. Note that clients do not have to perform the discovery process each time they intend to connect to a server. Typically, the information gathered during the discovery process is stored and reused for further connection establishment requests.

OPC UA specifies a set of abstract services for performing the discovery process as well as different design concepts. Section 9.4.2 describes the entities involved in discovery, and Sect. 9.4.3 shows how the services are used for performing the different discovery approaches.

### **9.4.2 *Discovery Entities***

For performing discovery, special entities are specified in [UA Part 12], each of them covering a special discovery boundary. The Discovery services are defined in [UA Part 4]. The entities are briefly introduced in the following and are the basis for understanding the discovery processes in OPC UA.

#### **9.4.2.1 Session Endpoint**

A Session Endpoint is always associated to an OPC UA server. Only this type of endpoints can be used for creating Secure Channels and Sessions to access data provided by the server.

#### **9.4.2.2 Discovery Endpoint**

Endpoints providing information about other endpoints are called Discovery Endpoints. This type of endpoint is either created by an OPC UA server providing data to clients or created by a Local or Global Discovery Server.

#### **9.4.2.3 Local Discovery Server**

If a discovery server resides on the same machine as the OPC UA servers of which it provides the Discovery Endpoints, then it is named as the Local Discovery Server. Multiple servers running on a single machine can share the same Local Discovery Server.

#### 9.4.2.4 Global Discovery Server

This discovery server maintains information about existing servers in a network and is accessible at a well-known address such as an URL or simply an IP address together with a port. More precisely, it provides the available Discovery Endpoints to which clients can connect to in order to get information about the Session Endpoints.

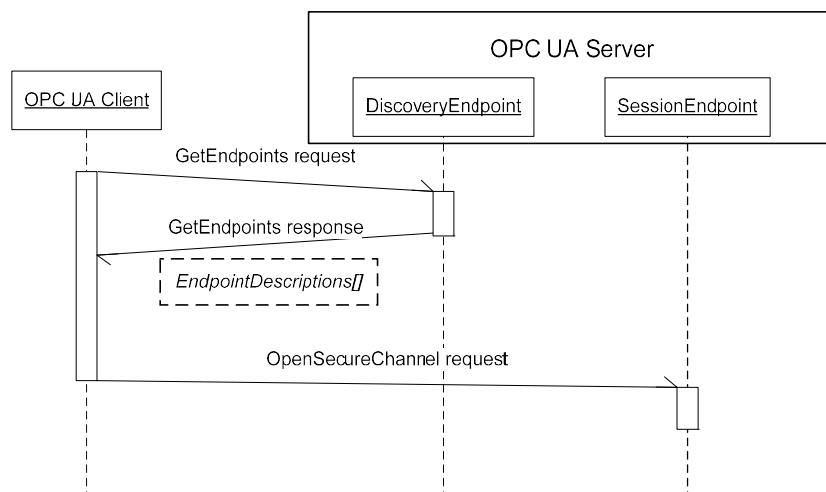
### 9.4.3 Discovery Process

Before clients can start the discovery process the installed servers (i.e. the endpoints) have to be registered in the discovery servers by using the RegisterServer Services defined in [UA Part 4]. Thereby a description and the discovery URL has to be provided.

The discovery process itself is performed by exchanging discovery messages among the above defined entities. There are a number of ways of how this process can be performed. The basic approaches are described in the following.

#### 9.4.3.1 Simple Discovery

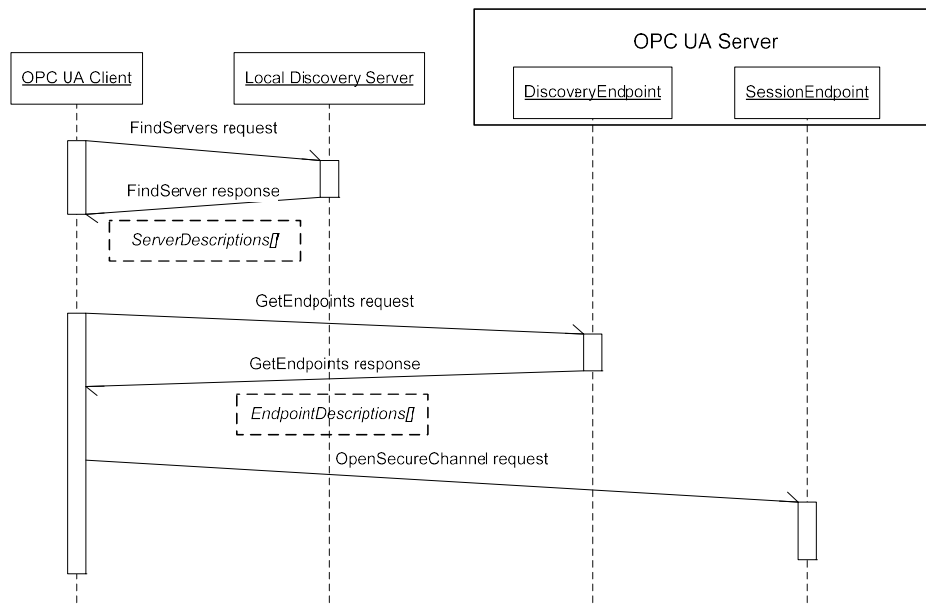
In the case that a client already has the address of the OPC UA server (e.g., by configuration), it only sends a GetEndpoints request to the server to get the descriptions of the available Session Endpoints. Once the client received the response from the Discovery Endpoints, it selects an appropriate Session Endpoint and establishes a connection to the selected endpoint starting with an OpenSecureChannel request. This is illustrated in Fig. 9.10.



**Fig. 9.10** Simple discovery

### 9.4.3.2 Normal Discovery

There will also be scenarios in which multiple OPC UA servers are running on a single machine. In such a case, the client asks the Local Discovery Server for the existing servers by sending a FindServers request. The address at which a client can access the Local Discovery Server is either well-known<sup>2</sup> or preconfigured in the client by the administrator. If the desired server is listed in the ServerDescriptions returned by the Local Discovery Server, the client extracts the DiscoveryURL and connects to the Discovery Endpoint of the server and proceeds like described in the Simple Discovery case. In Fig. 9.11, it is shown how Normal Discovery is performed.



**Fig. 9.11** Normal discovery

### 9.4.3.3 Hierarchical Discovery

If OPC UA servers are widely distributed at several locations within the network, it might be hard for a client to identify the desired server or even the machine it is running on. In such a case, a Global Discovery Server should be introduced providing information about these servers within the network. Thereby the client sends a FindServers request to a well-known (or preconfigured) Global Discovery Server and obtains the descriptions of the existing machines offering a Local Discovery Servers or OPC UA servers. If the returned list contains the desired OPC UA server, then the client proceeds as described in the Simple Discovery case,

<sup>2</sup>Port 4840.

otherwise it selects an appropriate Local Discovery Server running on a certain machine (e.g., the description may give hints that facilitates the decision which one to select) and proceeds like in the Normal Discovery described earlier. The complete message exchange for hierarchical discovery is depicted in Fig. 9.12.

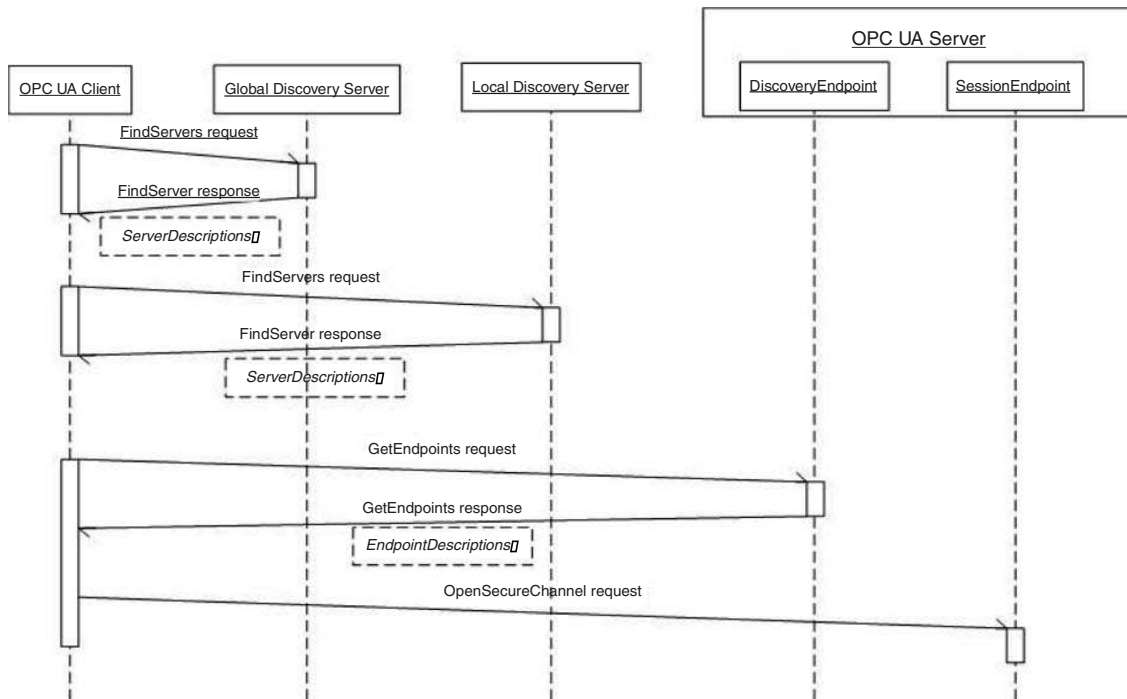


Fig. 9.12 Hierarchical discovery

## 9.5 Auditing

### 9.5.1 Overview

Auditing in the context of OPC UA means the tracking of activities of OPC UA applications, including normal and abnormal behaviors. It ensures thereby the traceability of the system for several purposes. One example is debugging applications on errors by extracting information from audit logs. Another example is forensic<sup>3</sup> activities after a security incident occurred. And in some special industries (e.g., consumer industries), it is even a hard requirement defined by regulatory bodies.

Auditing can be accomplished by one of the following approaches or by applying both of them:

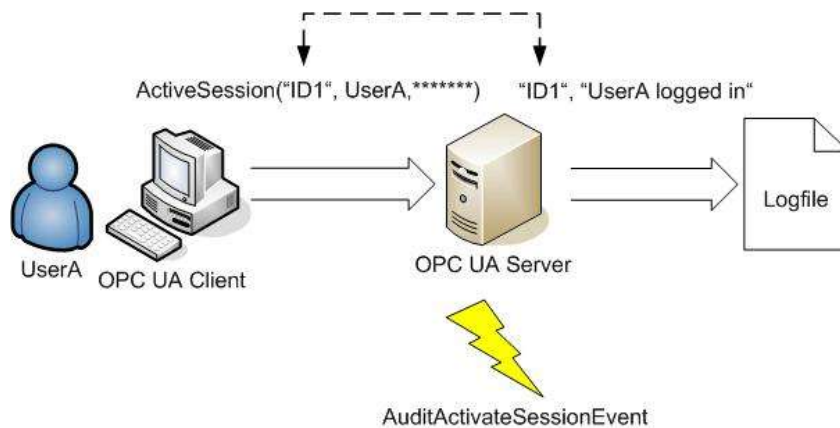
<sup>3</sup>Forensic in the context of security means gathering information that help to identify the reason why a certain security incident occurred (e.g., a hacker that intruded into a system). A typical activity is thereby exploring different log files to detect strange behaviors or anomalies.

- OPC UA applications generate audit events and store the audit information in logfiles or databases.
- OPC UA applications generate audit events and publish the audit event to which clients can subscribe. Thereby, clients can store audit information to special locations.

In the following, Sect. 9.5.2 describes the first approach and Sect. 9.5.3 describes the second one.

### 9.5.2 Audit Logs

Audit logs in OPC UA store information about certain events that occurred in an application. Each entry in an audit log has an identifier – the EntryID. An example of how audit logs can be used is illustrated in Fig. 9.13. In this scenario, UserA is logging in to the OPC UA server by using the ActivateSession Service. Therefore, an AuditActivateSessionEvent is generated by the server, which causes the creation of an audit log entry. The EntryID for the ActivateSession request is provided by the client in the request header of the message. If a client does not specify an EntryID in his request, the server uses an alternative identifier for the log entry. Note that also the client can create audit logs to track internal actions such as sending requests.

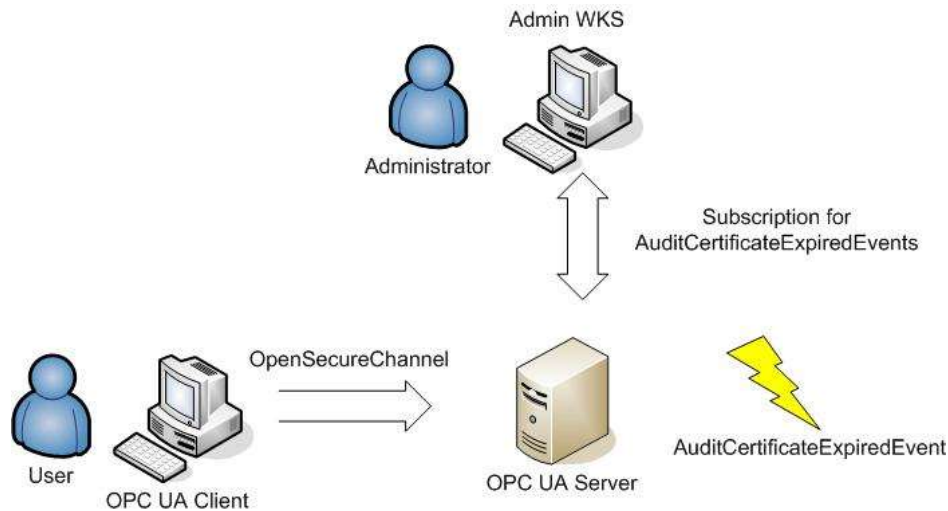


**Fig. 9.13** Example of how audit logs can be used

### 9.5.3 Audit Events

Audit events are different kinds of events occurring in a system for which a system should generate an audit log entry. OPC UA specifies a wide range of different audit event types that can be used directly or for subtyping a more specialized type. Clients can subscribe for audit events in the same way they subscribe for other events. An example is given in Fig. 9.14. In this example, during a Secure

Channel establishment it turns out that a certificate has expired. Therefore, an `AuditCertificateExpiredEvent` is created. Since the Admin workstation has also an OPC UA client, which subscribes for audit events of this type, it gets notified by the server. Now the administrator knows that certain clients identified with the `EntryID` need new certificates.



**Fig. 9.14** Example of how audit events can be used

### 9.5.4 Service Auditing

For each of the Service Sets defined in [UA Part 4] certain event types for auditing are defined and also how to deal with the events. For example, for the Secure-Channel Service Set the following special Event Types should be used:

- `AuditOpenSecureChannelEventType` for `OpenSecureChannel` Service
- `AuditCloseSecureChannelEventType` for `CloseSecureChannel` Service
- Subtypes of `AuditCertificateEventType` for certificate errors.

If the granularity of the types is not sufficient, they can easily be subtyped and additional ones can be defined. These EventTypes are used in both cases, when a service call fails and succeeds. For some services like for the services of the Discovery Service Set, no special EventTypes are defined. In this case, the base type `AuditEventType` can be used or a custom subtype if it.

A specific Profile is defining whether a server supports auditing by generating audit events.

### 9.5.5 Use Cases

This section describes how to handle audit logs and how they look like in certain system architectures (Sect. 9.2).

### 9.5.5.1 Client and Server Auditing

The first scenario describes the common client–server architecture shown in Fig. 9.15. When a client sends a request message to the server, then the client creates an audit log entry with the EntryID “CA”. When the server receives the message, it also creates an audit log entry, but with its own special EntryID “SB” and containing the client’s name and EntryID.

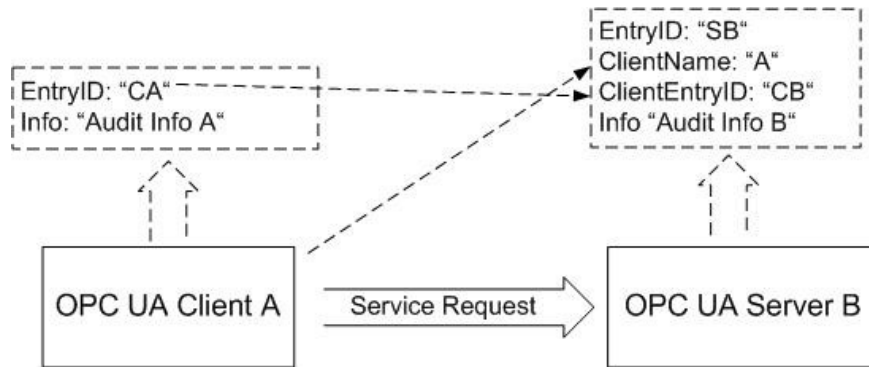


Fig. 9.15 Client–Server auditing

### 9.5.5.2 Aggregating Server

When considering a scenario in which an aggregated server is used as illustrated in Fig. 9.16. The first step is similar to the normal client–server approach described earlier. In the second step, the aggregating server sends a request to OPC UA Server C. When this server receives the message, it creates an audit log entry with the EntryID “SC” and the ClientName is thereby the name of the aggregating server “B” its EntryID “SB”.

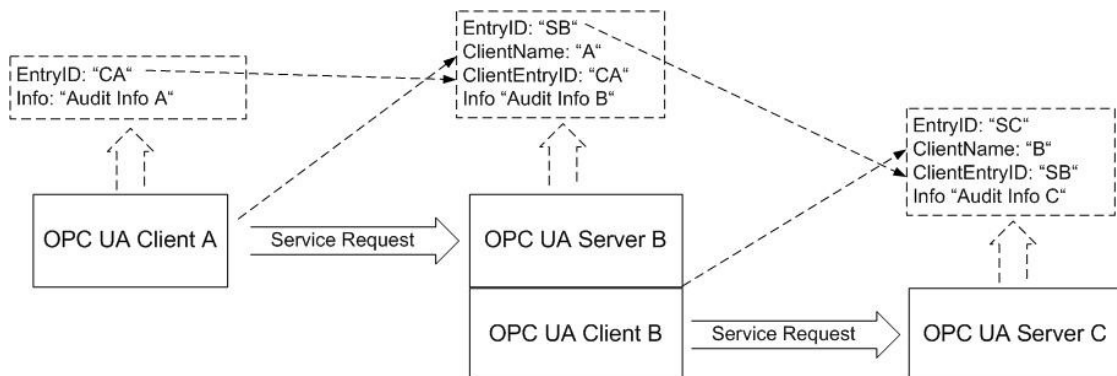
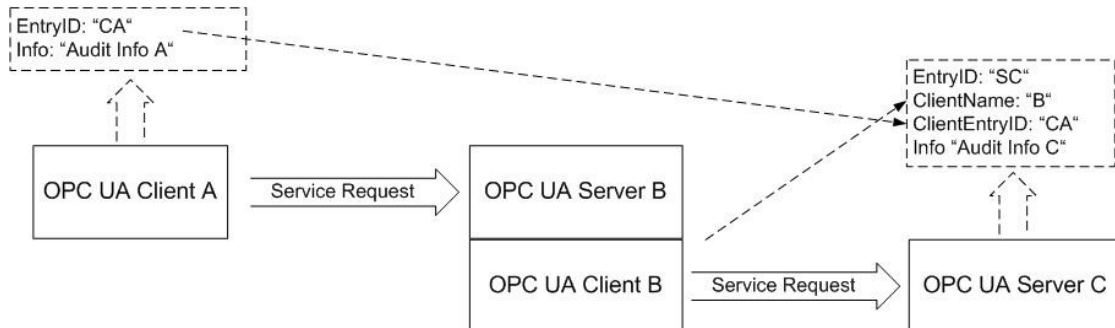


Fig. 9.16 Aggregating server auditing

### 9.5.5.3 Aggregation through a nonauditing server

The last use case considered in this section describes auditing in a scenario in which an aggregating server is used that does not support auditing. However, the

server it connects to does. Figure 9.17 depicts such a case. OPC UA Client A creates an audit log when it sends a request to the OPC UA Server B, which does not create an audit event but includes the EntryID in the request message it sends to Server C. When Server C receives the request from B, it creates an audit log entry with ClientName of “B” and the EntryID “CA” belonging to client A.



**Fig. 9.17** Aggregation through a nonauditing server

## 9.6 Summary

### 9.6.1 Key Messages

Applications based on OPC UA can be run at different levels in an automation network and is therefore applied for a diverse range of scenarios. However, there is a number of generic architectural patterns representing typical use cases or solving certain design problems. One problem domain in which some of the patterns can be applied is redundancy. OPC UA provides information about dealing with both client- and server-redundancy. On the server side, it is distinguished between transparent and nontransparent redundancy. Another important topic regarding the system architecture is discovery. Before an OPC UA client connects to a server, it first has to get information about how it can establish the connection. There are different possible configurations regarding communications protocols, encoding, and security. OPC UA specifies different ways of how discovery can be performed. When considering installing OPC UA applications in a concrete environment, it has to be taken care of how to configure and deploy those applications regarding discovery. Auditing is important also required for a number of reasons like for detecting errors or for accountability requirements defined by regulatory bodies. Therefore, OPC UA specifies how auditing is done with services and events.

### 9.6.2 Where to Find More Information?

The different concepts regarding redundancy in OPC UA are defined in [UA Part 4] as well as how services and events are audited. Some more general information

about auditing can also be found in [UA Part 2]. Special EventTypes for audit events are provided in [UA Part 3] and [UA Part 5]. There are two sources describing the discovery mechanisms of OPC UA: [UA Part 4] defines common services whereas [UA Part 12] comes up with different concepts of how the discovery process can be performed.

### ***9.6.3 What's Next?***

The next chapter describes how Classic OPC can be mapped to OPC UA. Thereby, it is shown how the different entities (i.e., objects and types) used in OPC Data Access, Alarms & Events, and Historical Access can be exposed in an OPC UA Address Space and also how they can be accessed.